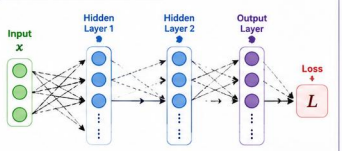
VANISHING GRADIENT PROBLEM

Why it happens and how modern activations solved it

1 THE NETWORK (Example)



Each neuron: $z = \sum w_i x_i + b \quad \rightarrow \quad o = \sigma(z)$

2 FORWARD PROPAGATION

1. **Weighted Sum** $z = \sum w_i x_i + b$ (Linear combination)
2. **Activation (Sigmoid)** $o = \sigma(z) = \frac{1}{1 + e^{-z}}$
3. **Pass to next layer** Repeat for all layers → get prediction $\hat{y}$

Loss: $L = \text{Loss}(y, \hat{y})$ (e.g., $(y - \hat{y})^2$

3 BACKPROPAGATION (Chain Rule)

We compute $\frac{\partial L}{\partial w}$ to update weights.

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial o_3} \times \frac{\partial o_3}{\partial o_2} \times \frac{\partial o_2}{\partial o_1} \times \frac{\partial o_1}{\partial w}$$

From Loss — Through Layers — To Weights

Update Rule (Gradient Descent)

$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w}$$

η = learning rate

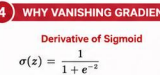
L = loss

4 WHY VANISHING GRADIENT HAPPENS (Due to Sigmoid)

Derivative of Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$



$0 \le \sigma(z) \le 1$

$0 \le \sigma'(z) \le 0.25$

Max derivative is 0.25 at $\sigma(z) = 0.5$

Multiplying Many Small Numbers

During backprop, many derivatives multiply.

$0.25 \times 0.25 \times 0.25 \times 0.25 \times \cdots$ (per layer)

After 5 layers: $0.25^5 = 0.000976$

After 10 layers: $0.25^{10} = 0.00000095$

After 20 layers: $0.25^{20} \approx 9.09 \times 10^{-13}$

Effect on Gradients

As we move backward, gradient becomes tiny.

Near Output (Larger) ← 0.1 → 0.025 → 0.00625 → 0.00156 → 0.00039 → ⋯ ← Near Input (Tiny!)

(× 0.25 each step)

Impact

- ✗ Gradients ≈ 0 in early layers
- ✗ Weights hardly update
- ✗ Learning is extremely slow or stops
- ✗ Model fails to capture patterns

So, $w_{new} \approx w_{old}$ (No learning!)

5 HOW IT WAS SOLVED: BETTER ACTIVATIONS

Sigmoid	$\sigma(z) = \frac{1}{1 + e^{-z}}$		✗ Saturates (0 or 1) ✗ Small gradients ✓ Causes vanishing gradient problem
Tanh	$\tanh(z)$		✓ Zero-centered ✓ Better than Sigmoid But still saturates
ReLU	$\text{ReLU}(z) = \max(0, z)$		✓ Gradient = 1 for $z > 0$ ✓ No saturation ($z > 0$) ✓ Faster training ✓ Greatly reduces vanishing gradient
PReLU	$f(z) = \max(\alpha z, z)$ (α is learnable)		✓ Improved ReLU ✓ Handles negative part ✓ Better performance
Swish	$f(z) = z \cdot \sigma(z)$		✓ Smooth & non-monotonic ✓ Strong empirical performance

In Short: Sigmoid's derivative is at most 0.25. Multiplying many such small values across layers makes gradients approach zero, preventing early layers from learning. Modern activations like ReLU, PReLU and Swish keep gradients healthy and enable training of very deep neural networks.

Key Takeaway:
The vanishing gradient problem is not just a math issue—
it shaped the entire evolution of Deep Learning!